

Practica 4.1

Aplicaciones Simple de Listas Perezosas

Datos del ejercicio:

```
datatype 'a seq = Empty  
| Cons of 'a * (unit -> 'a seq);
```

- Empty es secuencia vacía
- Cons(x, resto) guarda un valor x
- el resto no se calcula todavía
- el resto queda guardado como una función fn () => ...

también tengo estas funciones:

- seqFrom: genera naturales de forma perezosa
- takeSeq: saca los primeros n elementos y los convierte en lista normal
- dropSeq: descarta los primeros n
- mapSeq: aplica una función sin romper la pereza
- filterSeq: filtra sin generar todo de una vez

Entonces el mandato es:

1. construir la secuencia de múltiplos de un número
2. construir la secuencia de factoriales
3. construir la secuencia de Fibonacci
4. justificar cada una con:
 - cómo la hiciste
 - eficiencia frente a la versión recursiva normal
 - qué papel juega la evaluación diferida

```
SML
1  datatype 'a seq = Empty | Cons of 'a * (unit -> 'a seq);
2
3  fun seqFrom i = Cons(i, fn () => seqFrom(i + 1));
4
5  val natNum = seqFrom 0;
6
7  fun takeSeq (0, _) = []
8    | takeSeq (_, Empty) = []
9    | takeSeq (i, Cons(n, xt)) = n :: takeSeq(i - 1, xt());
10
11 fun dropSeq (0, xs) = xs
12   | dropSeq (_, Empty) = Empty
13   | dropSeq (i, Cons(_, xt)) = dropSeq(i - 1, xt());
14
15 fun mapSeq f Empty = Empty
16   | mapSeq f (Cons(v, xt)) = Cons(f v, fn () => mapSeq f (xt()));
17
18 fun filterSeq p Empty = Empty
19   | filterSeq p (Cons(n, xt)) =
20     if p n then Cons(n, fn () => filterSeq p (xt()))
21     else filterSeq p (xt());
```



```
Output
> val Empty = Empty: ∀ 'a . 'a seq;
> val Cons = Cons: ∀ 'a . 'a * (unit → 'a seq) → 'a seq;
> datatype 'a seq = {
  con Empty = Empty: ∀ 'a . 'a seq;
  con Cons = Cons: ∀ 'a . 'a * (unit → 'a seq) → 'a seq;
};
> val seqFrom = fn: int → int seq;
> val natNum = Cons (0, fn): int seq;
> val takeSeq = fn: ∀ 'a . int * 'a seq → 'a list;
> val dropSeq = fn: ∀ 'a . int * 'a seq → 'a seq;
> val mapSeq = fn: ∀ 'a 'b . ('a → 'b) → 'a seq → 'b seq;
> val filterSeq = fn: ∀ 'a . ('a → bool) → 'a seq → 'a seq;
```



```
23 fun multSeq k = mapSeq (fn n => n * k) (seqFrom 1);
24
25 fun factSeqFrom (n, acc) =
26   Cons(acc, fn () => factSeqFrom(n + 1, acc * (n + 1)));
27
28 val factorialSeq = factSeqFrom(1, 1);
29
30 fun fibSeqFrom (a, b) =
31   Cons(a, fn () => fibSeqFrom(b, a + b));
32
33 val fibonacciSeq = fibSeqFrom(1, 1);
34
35 val pruebaMult = takeSeq(6, multSeq 5);
36 val pruebaFact = takeSeq(6, factorialSeq);
37 val pruebaFib = takeSeq(8, fibonacciSeq);
> val multSeq = fn: int → int seq;
> val factSeqFrom = fn: int * int → int seq;
> val factorialSeq = Cons (1, fn): int seq;
> val fibSeqFrom = fn: int * int → int seq;
> val fibonacciSeq = Cons (1, fn): int seq;
> val pruebaMult = [5, 10, 15, 20, 25, 30]: int list;
> val pruebaFact = [1, 2, 6, 24, 120, 720]: int list;
> val pruebaFib = [1, 1, 2, 3, 5, 8, 13, 21]: int list;
```



A. Secuencia de múltiplos

```
fun multSeq k = mapSeq (fn n => n * k) (seqFrom 1);
```

Para la secuencia de múltiplos usé mapSeq sobre una secuencia infinita que empieza en 1. Así cada natural se transforma en $n * k$ y la solución queda corta y clara. En comparación con calcular cada múltiplo por separado, aquí ya queda montada la secuencia completa de forma ordenada. La evaluación diferida hace que solo se calcule el siguiente múltiplo cuando realmente se pide.



B. Secuencia de factoriales

```
fun factSeqFrom (n, acc) =  
  Cons(acc, fn () => factSeqFrom(n + 1, acc * (n + 1)));  
  
val factorialSeq = factSeqFrom(1, 1);
```

Para los factoriales usé un acumulador, porque cada término depende del anterior. Empecé en 1 y para obtener el siguiente multipliqué el resultado actual por $n + 1$. Eso es más eficiente que llamar una función recursiva de factorial para cada posición, porque evita repetir cálculos. La evaluación diferida permite que la secuencia sea infinita, pero solo se produzcan los factoriales que se vayan necesitando.



C. Secuencia de Fibonacci

```
fun fibSeqFrom (a, b) =  
  Cons(a, fn () => fibSeqFrom(b, a + b));  
  
val fibonacciSeq = fibSeqFrom(1, 1);
```

Para Fibonacci guardé dos valores consecutivos de la secuencia, porque cada nuevo término sale sumando los dos anteriores. Primero entrego a y luego continúo con la pareja $(b, a + b)$. Eso es mucho mejor que calcular $\text{fib } n$ con la recursión clásica en cada posición, ya que evita repetir demasiadas sumas. La evaluación diferida hace que solo avance al siguiente par cuando se piden más elementos.



➤ Explicación corta de las funciones dadas

```
datatype 'a seq = Empty | Cons of 'a * (unit -> 'a seq);
```

Define una secuencia perezosa. Tiene un valor actual y una cola que queda guardada como función para después.

seqFrom:

Genera i , $i+1$, $i+2$, ... sin crear todo de golpe.

takeSeq:

Toma los primeros n elementos de la secuencia y los convierte en una lista normal para poder verlos.



dropSeq:

Salta los primeros n elementos y deja el resto intacto.

mapSeq:

Transforma cada elemento, pero conservando la pereza.

filterSeq:

Se queda solo con los elementos que cumplen una condición, también de forma perezosa.

➤ **Justificación de perezosa**

La secuencia no se calcula completa desde el principio porque la cola está guardada como una función `fn () => ...`. Eso hace que el siguiente pedazo solo se construya cuando una función como `takeSeq` lo necesite. En SML eso se usa para simular evaluación perezosa, porque el lenguaje normalmente evalúa de forma eager.

